Two Edge Connected Graph

AlgoExpert · Very Hard · Graphs

The problem

- Undirected adjacency list; is the graph connected and free of bridges (stays connected after removing any single edge)?
- Empty graph counts as true.
- Tarjan low-link.

Intuition

- A graph has no bridges if every DFS subtree has a back edge reaching above its root.

Brute force vs optimal

- Remove each edge, test connectivity → O(E·(V+E))
- Tarjan's bridge-finding DFS → O(V + E)

Approach

- 1. DFS with arrival times
- 2. Compute the lowest reachable arrival time
- 3. A child with low time above the parent is a bridge
- 4. Check connectivity too

Worked example

```
2-edge-connected: connected, and no bridge

bridge = edge whose removal disconnects
```

Definition

Worked example

```
DFS: disc[v] = visit time

low[v] = earliest disc reachable via one back edge
```

Discovery and low times

Worked example

```
edge (v, w) is a bridge if low[w] > disc[v]

any bridge or unvisited vertex → false
```

Bridge test

Java solution (1/2)

```java
public class TwoEdgeConnectedGraph {
    public static boolean twoEdgeConnectedGraph(int[][] edges) {
        int n = edges.length;
        if (n == 0) return true;
        int[] disc = new int[n], low = new int[n];
        java.util.Arrays.fill(disc, -1);
        int[] time = {0};
        if (hasBridge(0, -1, edges, disc, low, time)) return false;
        for (int d : disc) if (d == -1) return false;
        return true;
    }
```

Java solution (2/2)

```
    private static boolean hasBridge(int v, int parent, int[][] edges, int[] disc, int[] low, int[] time) {
        disc[v] = low[v] = time[0]++;
        boolean skippedParent = false;
        for (int w : edges[v]) {
            if (w == parent && !skippedParent) { skippedParent = true; continue; }
            if (disc[w] == -1) {
                if (hasBridge(w, v, edges, disc, low, time)) return true;
                low[v] = Math.min(low[v], low[w]);
                if (low[w] > disc[v]) return true;
            } else low[v] = Math.min(low[v], disc[w]);
        }
        return false;
    }
}
```

Complexity

- Time: O(v + e)
- Space: O(v)

Edge cases & pitfalls

- Skip the parent edge once
- Disconnected → false
- Empty graph → true

Interview tips & follow-ups

- Relate to articulation points

Recap

- A graph has no bridges if every DFS subtree has a back edge reaching above its root.
- Complexity: O(v + e)